Test cases and its design Techniques:

- Introduction to test case and types
- Test case template
- How to write a test case and examples
- Preparing Review Report
- Software Test Life cycle
  - Writing Test Plan
  - Preparing Traceability Matrix
  - Writing Test Execution Report and Summary Report

****

Introduction to test case

After Software Development, we need to check each functionality in the product to make sure that actual requirements are fulfilled or not.

A **test case** is a set of conditions or actions that are performed on the software application to verify the expected functionality or feature of application is working as per business requirement. A Test Case is a script, program, a sequence of steps to validate whether a software application is working bugs free. A well-written test case should allow any tester to understand and execute the test.A collection of test cases is called a **Test Suite.** Different test suites might exercise different components or different types of functionality.

The benefits of an effective test case:

- To validate specific features and functions of the software.
- To help detect usability issues and design gaps early.
- Reduced software maintenance cost and time.
- Reusable test cases.
- Helps to verify that software meets the end user requirements
- Improved quality of software and user experience.

Test Case types:

Types of test cases can be defined as the variation on the usage of the test cases on each of testing stages in the software-testing life cycle.

1. Functional Test Cases
2. Performance Test Cases
3. Security Test Cases
4. Integration Test Cases
5. Positive Test Cases
6. Negative Test Cases
7. Database Test Cases
8. Acceptance Test Cases
9. Usability Test Cases

Test Case Type	Description	Test Step	Expected Result	Status
Functionality	Area should accommodate up to 20 characters	Input up to 20 characters	All 20 characters in the request should be appropriate	Pass or Fail
Security	Verify password rules are working	Create a new password in accordance with rules	The user's password will be accepted if it adheres to the rules	Pass or Fail
Usability	Ensure all links are working properly	Have users click on various links on the page	Links will take users to another web page according to the on-page URL	Pass or Fail

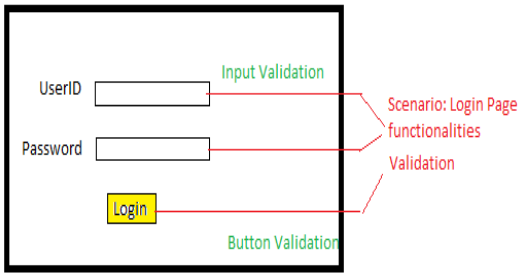
There are two types of test cases:

1. **Formal test cases:** Formal test cases are those test cases which are authored as per the test case format. It has all the information like preconditions, input data, output data, post conditions, etc. It has a defined set of inputs which will provide the expected output.
2. **Informal test cases:** Informal test cases are authored for such requirements where the exact input and output are not known.

Ex: Write test cases for verify login of Gmail account.

Here are some test cases.

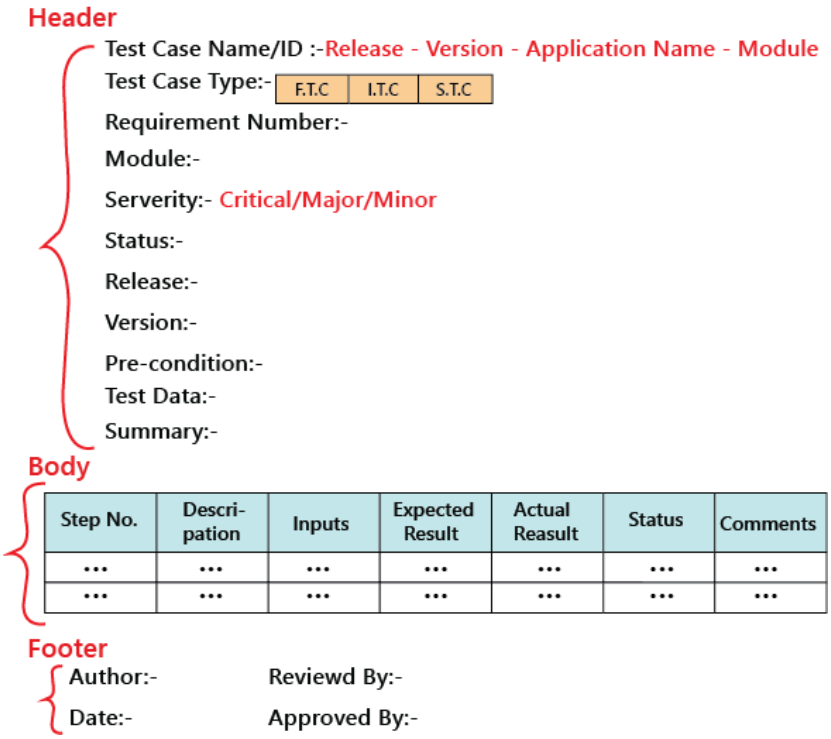
1. Enter valid User Name and valid Password
2. Enter valid User Name and invalid Password
3. Enter invalid User Name and valid Password
4. Enter invalid User Name and invalid Password



Test Case Template

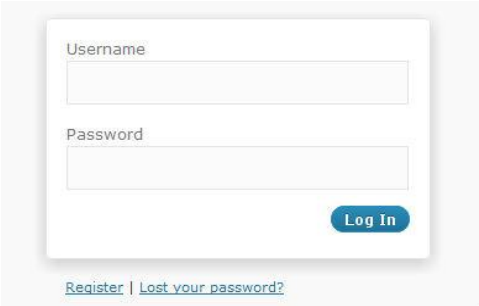
A Test Case Template is a well-designed document for developing and better understanding of the test case data for a particular test case scenario. This template is also known as a test case template.

A good Test Case template helps understanding the test cases. Writing test case in a standard format reduces the test effort and the error rate.



Test Case Field	Description
Project Name:	Name of the project the test cases belong to
Module Name:	Name of the module the test cases belong to
Test case ID:	Each test case should be represented by a unique ID. To indicate test types like "TC_UI_1" indicating "User Interface Test Case#1."
Name or Test Title/Test Case	Title of the test case
Test Scenario:	Test Scenario ID or title of the test scenario.
Test Priority:	Low,Medium, High
Name of the Module:	Determine the name of the main module or sub-module being tested
Test Designed by:	Name of the Tester who designed the test cases
Date of test designed:	Date When the test cases were created
Test Executed by:	Name of the Tester who executed the test case
Date of the Test Execution:	When the test case was executed
Date Of Review:	When the test cases were reviewed
Description of Test	Determine the purpose of test in detail
Pre-condition:	Conditions which needs to meet before executing the test case.
Reference Document:	Path of the reference documents such as Requirement Document, Test Plan, Test Scenarios etc.
Test Steps:	Procedure to execute the test. describe the steps to perform tests
Test Data:	Data required for execution of test. Data used an input for the test cases. That data needs to be prepared for testing
Expected Results:	The expected output of the test
Post-Condition:	Conditions which needs to achieve when the test case was successfully executed.
Actual Result:	After test execution, The actual output of the test
Status (Fail/Pass)	If the actual and expected results are same, mention it as Passed else Failed. If a test fails, it has to go through the bug life cycle to be fixed.
Test Environment	The environment (H/w,S/w,N/w) in which the test was executed.

Example: write test cases for Verify the login functionality of the Gmail account.



Here are some test cases.
1. Enter valid User Name and valid Password
2. Enter valid User Name and invalid Password
3. Enter invalid User Name and valid Password
4. Enter invalid User Name and invalid Password

Each test case should analyze to determine how a particular condition is satisfied or not. If the condition is satisfied the Test Status is Passed, Else Failed.

Project Name	Email Application
Module Name	Login Module
Created By	Nayantara
Date of Creatio	DD-MM-YYYY
Date of Review	DD-MM-YYYY

Test Case Id	Test Scenario	Test Case	Pre-Condition	Test Steps	Test Data	Expected Result	Post Condition	Actual Result	Status
TC_Login_001	Verify the Login of Mail	Enter Valid User Name and Valid Password	1. Need a Valid Mail Account to login	1.Enter User Name 2.Enter Password 3. Click "Login" Button	<Valid User Name> <Valid Password>	Sucessful Login	Mail Inbox is shown	Sucessful Login	Pass
TC_Login_002	Verify the Login of Mail	Enter Valid User Name and Invalid Password	1. Need a Valid Mail Account to login	1.Enter User Name 2.Enter Password 3. Click "Login" Button	<Valid User Name> <Invalid Password>	Login Failed	A error message "Entered Login details Incorrect "	Sucessful Login	Fail
TC_Login_003	Verify the Login of Mail	Enter Invalid User Name and Valid Password	1. Need a Valid Mail Account to login	1.Enter User Name 2.Enter Password 3. Click "Login" Button	<Invalid User Name> <Valid Password>	Login Failed	A error message "Entered Login details Incorrect "	Login Failed	Pass
TC_Login_003	Verify the Login of Mail	Enter Invalid User Name and Invalid Password	1. Need a Valid Mail Account to login	1.Enter User Name 2.Enter Password 3. Click "Login" Button	<Invalid User Name> <Invalid Password>	Login Failed	A error message "Entered Login details Incorrect "	Login Failed	Pass

How to write a Test Case

Two ways of writing test case through which tester can verify the functionalities of the software.

- Positive Test Cases: to verify the application behavior on positive test data input.
- Negative Test Cases: to verify the system behavior on negative or invalid test data input.

1. Identify the scope and purpose of testing

You have to understand the purpose of testing, features and identify the testable user requirements

2. Define how to perform testing activities

You should have better understanding of functional and non-requirements of an application. You must know about what all operations can be used. Test cases can be written based on requirements.

3. Avoid test case repetition.

Do not repeat test cases. If a test case is needed for executing some other test case, call the test case by its test case id in the pre-condition column. Use unique Test Case ID

4. Define the test case framework

Test case framework can be designed depend on systems requirements. Test cases should cover functionality, compatibility, UI interface, fault tolerance, and performance.

5. Preparation of test data for variety of test cases

Variety of test data is required for executing the test cases like valid data to test positive test cases, invalid data to test negative test cases, legal invalid data to test boundary value conditions, illegal and abnormal data to test error handling and recovery test cases

6. Structuring of test cases

Required all the necessary information to writing test cases. We have detailed preparation list of test scenarios from the requirements. Each test scenario has one or multiple test cases.

7. Test Cases need to be simple, transparent and easy to understand.

Test cases should be detailed and clear, new tester can easily execute the test cases. The purpose and scope of test cases should be well defined.

8. Test cases should be valid, brief and short

Test cases should have only necessary and valid steps. If single test case having too many test steps to

be performed then this may lose concentration & aim of test case. So, single test case should have one expected result and not try to cover too many expected results.

9. Test Cases Should Be Traceable

Each test case should be traceable and it should be linked to "Requirement ID", which helps to check the 100% coverage of requirements. No any requirement is missed to test.

10. Test cases should be reusable & maintainable:

A well written test case is reusable and maintainable. When developers change the code, then testers need to update the test cases. If test cases were easy to understand then it would be easy to update.

- **Write Test Cases for Registration and Login Page**

You should gather all the customer requirements you can. The list of requirements which can be useful are:-

Registration Form having following fields:-

1. FirstName 2. LastName 3. UserName 4. Email Id
5. Password 6. Confirm Password 7. Address 8. PhoneNumber
9. Gender

Required fields are: 1. EmailId 2. UserName 3. Password 4. Confirm Password

Validation for fields:

1. Email
2. Password: It should have alphanumeric; Length should be 8 to 32
3. PhoneNumber: Phone Number should have only Numbers, Country code is required

— After successful Verification email should send to the user

- **Test case for registration page**

Signup page and login page by assuming some client requirements such as:-

- Username and password are mandatory fields
- There is a cancel and reset button at the bottom of the form
- Radio buttons and checkboxes are placed correctly
- The limit of Password should be 8-13 characters (alphanumeric).

Sr. No	Test Cases	Unit / Module	Description	Test Steps	Test Data/Input	Expected Results
1	TC-001	User Interface	Check all the text boxes, radio buttons, etc	1. Click on Radio buttons, buttons and dropdowns		UI should be perfect
2	TC-002	Required fields	Check required fields by not filling any data	1. Do not enter any value in the field 2. Click on Signup button	Nil	It Should show mandatory symbol (*) on mandatory fields.
3	TC-003	Required fields	Check the required fields by filling the data	1. Enter valid values in required fields 2. Click on Signup button	Nil	1. User should be register successfully 2. A successful registration message should show 3. Mail should send to the user
4	TC-004	Optional fields	Check all the optional field when do not fill data	1. Do not enter any detail in optional fields 2. Enter valid data in required fields 3. Click on Signup button	Nil	1. It should not ask to fill the optional fields 2. User should be register successfully 3. A successful registration message should show 4. Mail should send to the user
5	TC-005	Optional fields	Check all the optional field when fill data	1. Enter valid data in optional fields 2. Enter valid data in required fields 3. Click on Signup button	Nil	1. User should be register successfully 2. A successful registration message should show 3. Mail should send to the user

6	TC-006	Email validation	— Check Email text field that has Email address without @ symbol. — Check Email text field that has random string instead of real email. — Check Email text field that has @ symbol written in words. — Check Email text field that has missing dot in the email address.	1. Enter Invalid Emails 2. Click on Signup Button	1. testAtgmail.com 2. test@gmailcom 3. test@gmail 4. @gmail	It should show the validation message for valid email
7	TC-007	Email validation	Check all the valid emails	1. Enter Valid Emails 2. Click on Signup Button	1.test@192.168.1.1 2. test@gmail.com	It should not show any validation message
8	TC-008	Phone Number validation	Check the phone number when pass alphanumeric data	1. Enter alphanumeric data in phone field 2. Click on signup button	1. dada5$7567#7	It should show the validation message for Phone Number
9	TC-009	Phone Number validation	Check the phone number when not pass country code	1. Enter valid phone number without country code 2. Click on signup button	1. 9012078654	It should show the validation message for country code is required
10	TC-010	Phone Number validation	Check the phone number when pass country code	1. Enter valid phone number with country code 2. Click on signup button	1. +9190112244	It should not show any validation message
11	TC-011	Password validation	Check the password limit when enter value less than min	1. Enter value which is alphanumeric but less than 8 2. Click on signup button	1. Password	It should show validation message
12	TC-012	Password validation	Check the password limit when enter value greater than max	1. Enter value which is alphanumeric but less more than 32 2. Click on signup button	1. redhatisgood434adasdasdasd3243dsadadsad432423	It should show validation message
13	TC-013	Password validation	Check the password when pass only numbers	1. Enter value in numbers which is in between 8-32 2. Click on signup button	1. 12345678	It should show validation message
14	TC-014	Password validation	Check the password when pass valid data	1. Enter value in alphanumeric which is in between 8-32 2. Click on signup button	1. Pass123456	It should not show any validation message Test case for Login page

Test case for login page

Sr. No.	Test Cases id	Module	Test Case	Test Steps	Expected Results
1	TC-001	User Interface	Check all the text boxes, and buttons	Check page	UI should be perfect. Text boxes and button should be aligned
2	TC-002	User Login	Check When pass invalid user name	1. Enter invalid username 2. Enter correct password 3. Click on Login Button	User should not login and should show proper error message
3	TC-003	User Login	Check When pass correct username and invalid password	1. Enter valid username 2. Enter incorrect password 3. Click on Login Button	User should not login and should show proper error message
4	TC-004	User Login	Check Keeping Password blank	1. Enter valid username 2. Do not enter password 3. Click on Login Button	User should not login and should show proper error message

5	TC-005	User Login	Check when pass correct email and password	1. Enter valid username 2. Enter valid password 3. Click on Login Button	— User should login
6	TC-006	User Login	Check if password is entered in encrypted form	1. Enter valid username 2. Enter password 3. Click on Login Button	Password is entered in encrypted form
7	TC-007	Signup for new users	Check whether the signup link for new user is working	Click Signup link	Clicking signup link takes user to signup page successfully

Preparing Review Report

Review is generally performed by:

1. **Self-review:** It is done by the tester himself who has written the test cases. He can verify whether all the requirements are covered or not by looking into SRS/FRD.
2. **Peer review**: It is done by another tester who hasn't written those test cases but is familiar with the system under test. Also known as Maker and Checker review.
3. **Review by a supervisor:** It is done by a team lead or manager who is superior to the tester who has written the test cases and has great knowledge about the requirements and system under test.

Test case review processes are:

1. ***Spelling mistakes:*** it can create many confusions or make a sentence difficult to understand.
2. ***Grammar:*** If grammar of test cases is not proper, then interpreting may be wrong results.
3. ***Template format:*** Proper template becomes easy to add/modify test cases in future and test case plan looks organized.
4. ***Standard/Guidelines:*** While review process to check whether all the standards and guideline are properly followed.
5. ***Language used:*** Test cases should have a very simple language, which is easy to understand.
6. ***Functionality coverage:*** all functionality associated with the system under test should be covered
7. ***Replication:*** It refers to the duplicate test cases removal. It is possible that two or more test cases test the same. They can merged into one, this would save time and space.

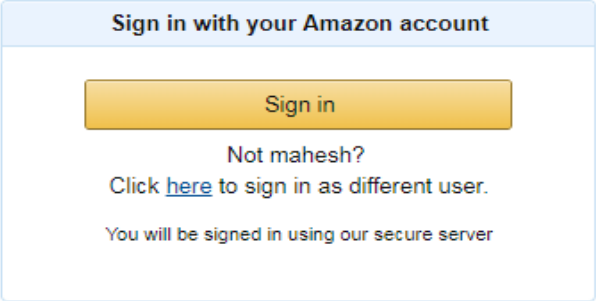
Sample Template for Test Case Review

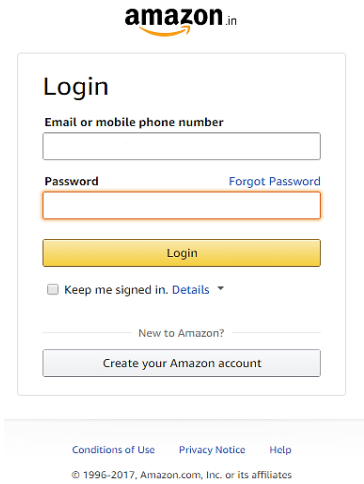
Test Case ID: _____ Reviewed By: _____ Date of Review: _____ Version of Review: _____				Review Comments Incorporated By : _____ Date Incorporated: _____ New Version of Document: _____		
Sr.	Page No.	Step No.	Severity C – Critical, M – Major, S – Small	Comments	Action Taken	Close (Yes / No)
01						
02						
03						
04						
05						
06						
07						
08						
09						
10						

Examples On Writing Test Cases

2) Amazon Login Scenario

When you already login to the Amazon and revisit the website. It will show you special box that shows you are already identified. And it requires you to enter password to next page. And for that flow, it only shows sign in box as shown below.





Positive Test Cases

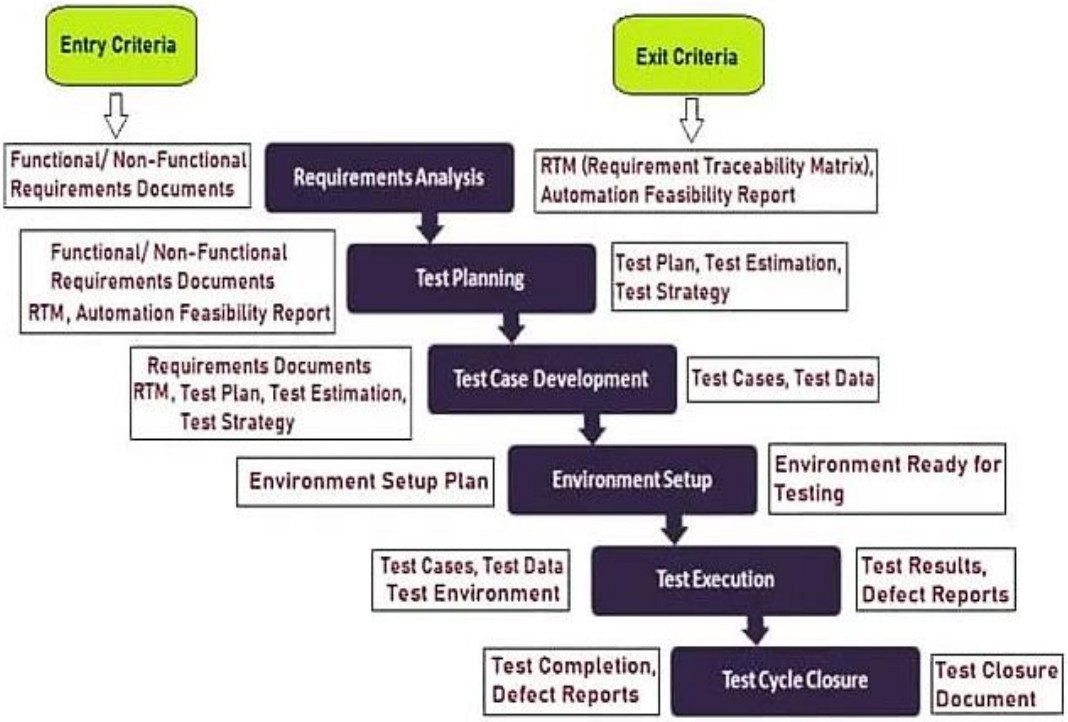
- Enter Valid Username.
- Enter Valid Password.
- Click on "Forgot Password" Link and find out forgot password page.
- Click on Create new account button and verify if it takes to new account page.
- Select "Keep me signed in" option to retain login session.

Negative Test Cases

- Enter invalid username and valid password.
- Enter valid username and invalid password.
- Enter invalid username and password.
- Keep one field blank either username or password and check login.

Software Test Life Cycle (STLC)

The Software Testing Life Cycle (STLC) is a sequence of specific actions performed during the testing process to ensure that the software quality objectives are fulfilled. The STLC includes both verification and validation. Each phase is carried out in a planned and systematic manner in order to meet the quality goals. Each step has clear goals, actions, and deliverables that input to the next step. The different phases of Software testing life cycle are:



Each of the steps have:

- **Entry Criteria:** It defines precondition that must be completed before testing can start
- **Exit Criteria:** It defines conditions that must be fulfilled before testing can be stop.

1. Requirement Analysis

- In this phase QA team understands the requirements like functional and non-functional and whether these requirements are testable or not.

- If anything is missing or not understandable then QA team communicate with the stakeholders to better understand the requirements in details.
- Once QA team is clear with the requirements they will documented acceptance Criteria and get it approved by the Customers.
- Testers analyze customer requirements and work with developers during the design phase to see which requirements are testable and how testing going to test those requirements.
- Defect fixing cost is very less if defect found in requirements phase rather than in next phases.

Activities to be done	Deliverables (Outcome)
Entry Criteria: client requirements, acceptance criteria, and planned product architecture to be documented.	**Exit Criteria:** aim for a requirement traceability matrix (RTM) and decision of automation.
▪ Gather details and identify customer needs with testing priorities ▪ What types of testing to be performed? ▪ Prepare Requirement Traceability Matrix ▪ Identify test environment details ▪ Automation Feasibility Analysis.	▪ Requirement Traceability Matrix (RTM) ▪ Automation feasibility report

2. Test Planning and Control(Test Strategy)

- In this phase QA manager prepare and finalize detailed test plan based on the requirement analysis.
- A test plan determining which needs to be tested, how testing will be done, testing strategy, test environment, automation tools, and resource planning, risks etc.
- Testing team calculates estimated effort and cost for the testing work.
- This is very important phase as any small mistake in this phase can lead to major issues in the project regarding time, money, efforts, etc.

Activities to be done	Deliverables (Outcome)
Entry Criteria: Report the implemented test strategy.	**Exit Criteria:** Testers should have approval on the test plan's risks and costs.
▪ Preparation of Test Plan, Test strategy documents ▪ Test tool selection and testing approach ▪ Test effort estimation. ▪ Resource planning and determining roles and responsibilities. ▪ Training requirement.	▪ Test Plan document ▪ Test Strategy document ▪ Number of Resources used ▪ Test effort and cost estimation report

3. Test Case Development

- In this phase, a testing team prepare detailed test cases for requirement, test automation scripts, and test data. All prepared test cases are reviewed or updated by peer members or QA lead.
- Each Test case defines test inputs, procedures, execution, conditions, and expected results.
- Test cases should be transparent, efficient, and adaptable. Ensure 100% coverage of software requirements.
- The Requirement Traceability Matrix maps the requirements with test cases. It helps to verifying that test cases have been written for all requirements.

Activities to be done	Deliverables (Outcome)
Entry Criteria: Approval of timelines to execute the test plan.	**Exit Criteria:** Approval of test cases and automation scripts.
▪ Creation of test cases and automation scripts ▪ Review test cases and scripts. ▪ Create test data	▪ Test cases/Scripts ▪ Test data ▪ RTM

4. Test Environment setup

- In this step, the QA team sets up the software environment based on the hardware and software specifications. If the software is multi-platform, the team will create multiple such environments.

- Once the environments are successfully deployed, smoke testing can be performed to ensure the environment is working with the intended functionality and would also provide confidence in testing the new feature.
- This activity can be started parallel with Test Case Development independently.

Activities to be done in Test Environment Setup phase are given below:

- As per the Requirement and Architecture document the list of required software and hardware is prepared
- Setting up of test environment
- Creation of test data
- Installation of build and execution of Smoke testing on it

Deliverables (Outcome) of Test Environment Setup phase are:

- Test Environment setup is ready
- Test Data is created
- Results of Smoke testing

5. Test Execution

- Before starting the Test Execution phase the Test Case Development and Test Environment setup should be ready.
- In Test Execution phase the test cases are executed in the testing environment.
- In this phase testing team start executing test cases based on prepared test planning & prepared test cases in the prior step.
- While execution of the test cases the QA team may find bugs which will be reported against that test case. This bug is fixed by the developer and is retested by the QA.
- Once the test case is passed then same can be marked as Passed. If any test case is failed then corresponding defect can be reported to developer team via bug tracking system & bug can be linked for corresponding test case for further analysis.
- Every failed test case should be associated with at least single bug. Once the bug fixed by development team then same test case can be executed based on your test planning.
- If any of the test cases are blocked due to any defect then such test cases can be marked as Blocked, so we can get the report based on how many test cases passed, failed, blocked or not run etc. Once the defects are fixed, same Failed or Blocked test cases can be executed again to retest the functionality.

Activities to be done in Test Execution phase are given below:

- Execution of Test Cases
- Reporting test results
- Logging defects for the failed test cases
- Verification and retesting of the defect
- Closure of defects

Deliverables (Outcome) of Test Execution phase are:

- Test execution Report
- Updated test cases with results
- Bug Report

6. Test Cycle Closure

- In order to start the Test Cycle Closure activity the Test Execution phase should be completed. In Test Cycle phase the QA team will meet and discuss about the testing piece.
- The whole goal of this discussion is to learn lessons from the bad practices. This will help in future projects.
- Call out the testing team member meeting & evaluate cycle completion criteria based on Test coverage, Quality, Cost, Time, Critical Business Objectives, and Software.
- Discuss what all went good, which area needs to be improve & taking the lessons from current STLC as input to upcoming test cycles, which will help to improve bottleneck in the STLC process.

- Test case & bug report will analyze to find out the defect distribution by type and severity.
- Once complete the test cycle then test closure report & Test metrics will be prepared. Test result analysis to find out the defect distribution by type and severity.

Activities to be done in Test Cycle Closure phase are given below:

- To evaluate the test completion on the basis of Test Coverage and Software Quality
- Documentation of the learning from the project
- Analyzing the test results to find out the distribution of severe defects
- Test Closure Report preparation

Deliverables (Outcome) of Test Cycle Closure phase are:

- Report of Test Closure

Writing Test Plan

A test plan is a detailed document that outlines the test strategy, Testing objectives, resources (manpower, S/W, H/W) required for testing, test schedule, Test Estimation and test deliverables. The test plan provide a blueprint to conduct software testing activities process, which is monitored and controlled by the test manager. Test plans outline the process of testing the functionality of software. A test plan details each step taken to achieve a certain result and states the objective of each action. The plan also highlights the projected resources, risks, and personnel involved in the test. You should use a test plan if you are seeking to eliminate bugs and other errors in your software before it becomes available to customers.

➢ Test Plan types

- **Master Test Plan:** A high-level plan, which includes all others.
- **Testing Level Specific Test Plans:** Plans for different testing levels- Unit testing, Integration testing, System testing, Acceptance testing
- **Testing Type Specific Test Plans:** Plans for types of testing as Performance Test Plan and Test Plan Security.

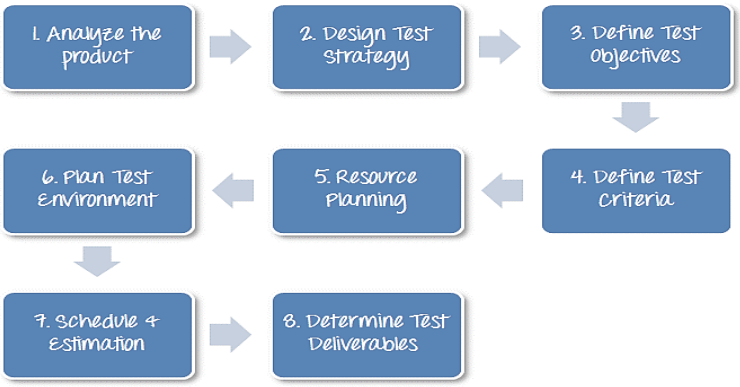
➢ Importance of Test Plan

Making Test Plan has multiple benefits

- Test Plan helps us to determine the effort needed to validate quality of theapplication under test
- Helps to developers, business managers, customers understand the details of testing.
- Test Plan guides our thinking. It is like a rule book, which needs to be followed.
- Test estimation, test scope, Test Strategy are documented in Test Plan, so it can be reviewed by Management Team and re-used for other projects.

➢ How to write a Test Plan

1. Analyze the product
2. Design the Test Strategy
3. Define the Test Objectives
4. Define Test Criteria
5. Resource Planning
6. Plan Test Environment
7. Schedule & Estimation
8. Determine Test Deliverables



⦿ Step 1) Analyze the product

How can you test a product without any information it is Impossible. You must learn a product carefullybefore testing it. You should research clients and the end users to know their needs and expectations from the application

- Who will use the website?
- What is it used for?
- How will it work?
- What are software/ hardware the product uses?

⦿ Step 2) Develop Test Strategy

A Test Strategy document is a high-level document, which is usually developed by Test Manager.

This document defines:

- The project's testing objectives
- Determines testing effort and costs

Step 2.1) Define Scope of Testing: Before starting test activity, scope of the testing should be known.

1) In Scope: Scope defines the features, functional or non-functional requirements of the software that will be tested.The components of the system to be tested (h/w, s/w, middleware etc)

2) Out of Scope: clearly defines the features, functional or non-functional requirements of the software that will not be tested. Defining accurate scope of testing helps you-

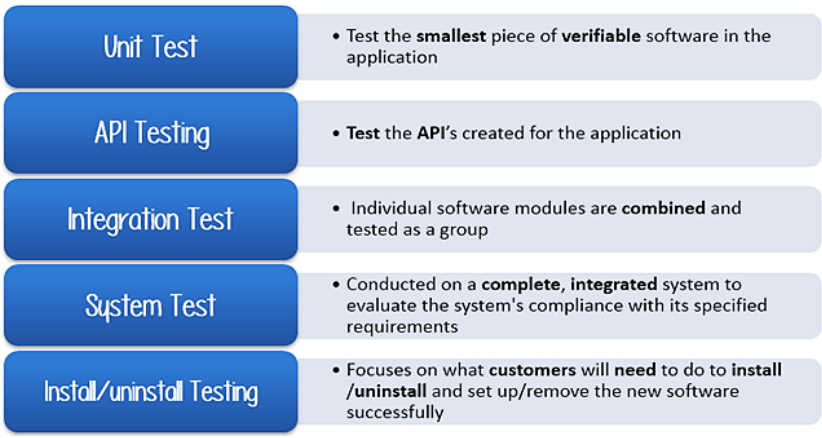
- Give everyone a confidence & accurate information of the testing you are doing
- All project members will have a clear understanding about what is tested and what is not

How do you determine scope your project?

- accurate customer requirement
- Project Budget
- Product Specification
- Skills & talent of your test team

You should clearly define the "in scope" and "out of scope" of the testing.

Step 2.2) Identify Testing Type: A Testing Type is a standard test procedure that gives an expected test results. Each testing type is planned to identify a specific type of product bugs. But, all Testing Types aim is achieving one common goal "Early bug detection before releasing the product to the customer"



There are number **of Testing Types** but your testing team **cannot have** enough efforts to handle all kind of testing. As Test Manager, you must set **priority** of the Testing Types

- Which Testing Types should be **focused** for web application testing?
- Which Testing Types should be **ignored** for saving cost?

Step 2.3) Document Risk and Issues (Risk Management): Risk is future's uncertain event with a probability of loss. When risk actually occurs, it becomes '**issue'.**

List out the risks involved in testing process. Along with risks write mitigation &contingency actions if risk occurred.

Some basic rules to follow at the time of developing test plan:

- Keep your plan flexible so that it can be enhanced later for adding test cases, test data etc. Change control mechanism should be there to track changes.
- Test plan should be reviewed frequently before base lining.
- Test plan should be short, readable because test plan act as a communication document.

Risk	Mitigation
Team member have lack of skills for website testing.	Plan **training course** to skill up your members
The project schedule is too tight; it's hard to complete this project on time	Set **Test Priority** for each of the test activity.

Test Manager has poor management skill	Plan **leadership training** for manager
A lack of cooperation negatively affects your employees' productivity	**Encourage** each team member in his task, **and inspire** them to greater efforts.

Step 2.4) Create Test Logistics: Test Manager should answer the following questions:

Who will test?

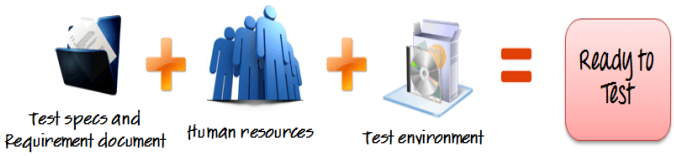
To select the right member for specified task if his skill is qualified for the task or not, also estimate the project budget. Type of tester can be defined. Selecting wrong member for the task may cause the project to **fail** or **delay**.

Person having the following skills:

- Ability to **understand** customers point of view
- Strong **desire** for quality
- **Attention** to detail
- Good **cooperation**

When will the test occur?

Test activities must be matched with associated development activities. You will start to test when you have **all required items** shown in following figure



⦿ Step 3) Define Test Objective

Test Objective is the overall goal and achievement of the test execution. The objective of the testing is finding as many software defects and ensure that the software is **bug free** before release.

To define the test objectives:

1. List all the software features (functionality, performance, GUI) which may need to test.
2. Define the target or the goal of the test based features
3. Ensure the Application Under Test conforms to functional and non-functional requirements
4. Ensure the AUT meets the quality specifications defined by the client
5. Bugs/issues are identified and fixed before go live

⦿ Step 4) Define Test Criteria

Test Criteria is a standard or rule on which a test procedure or test judgment can be based.

Two types of test criteria:

1. Test Suspension & Resumption Criteria:

Test suspension criteria should explain when to pause/suspend the testing activities.

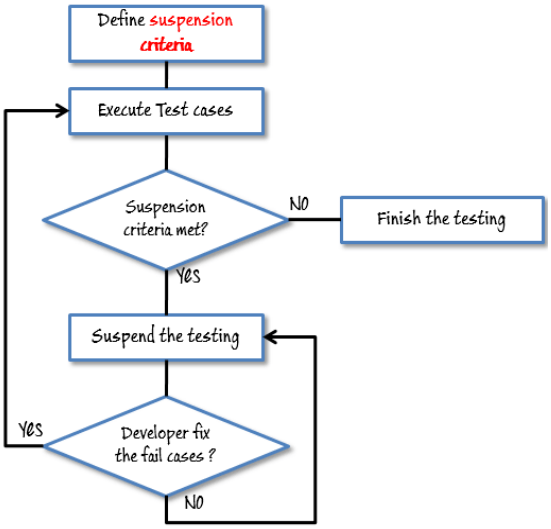
Test resumption criteria explains when to continue with testing activities after resolving issues.

Test team need to suspend or terminated the testing because of some major enhancements in project.If the suspension criteria are met during testing, the active test cycle will be **suspended** until the criteria are **resolved**.

Example: In your team members report shows there are **40%** of test cases failed, you should **suspend** testing until the development team fixes all the failed cases.

2.Test Exit Criteria

A test exit criterion expresses when to stop testing. Factors like release deadlines, test coverage, test budget reduction, confidence in test executed. Example: **95%** of all critical test cases must be passed.



Some methods of defining exit criteria are

- **Run rate** is ratio between **number test cases executed/total test cases** of test specification.
- Example: total 120 TCs, but the tester only executed 100 TCs, so the run rate is 100/120 = 0.83 (83%)
- **Pass rate** is ratio between **numbers test cases passed / test cases executed**.
- Example: 100 TCs executed, there are 80 TCs passed, so the pass rate is 80/100 = 0.8 (80%)

Your Team reported test execution result to you and they want to confirm the **Exit Criteria.** In case, the Run rate is mandatory is **100%,** but the test team only completed 90% of test cases. It means the Run rate is not satisfied, so do not confirm the Exit Criteria.

⦿ Step 5) Resource Planning

Resource plan is a **detailed summary** of all resourcestypes required to complete project task. Resource could be human, equipment and materials needed to complete a project.The resource planning is important factor of the test planning because helps in determining thenumber of resources (employee, equipment etc.) to be used for the project. Test Manager can make the correct schedule & estimation for the project. Detail description of the Roles and responsibilities of different team members are for each stage of testing activity.

1. Human Resource: The following table represents various members in your project team

Member	Tasks
Test Manager	Test Managers responsibility is to plan & manage testing activities, preparing reports etc. Deliverables includes test plan, test cases/Scripts, defect logs, reports, project directions
Tester	Tester's responsibility is to execute tests, defect logging & tracking.
Test Administrator	Builds up and ensures Test Environment and resources are managedandmaintained. Support Tester to use the test environment for test execution
SQA members	Check to confirm whether the testing process is meeting specified requirements

2. System Resource

Resources	Descriptions
Server	web server, database server, and application server
Test tool	test tools are Selenium, QTP etc.
Network	Network devices include LAN and Internet
Computer	The PC which users often use to connect the web server

⦿ Step 6) Plan Test Environment

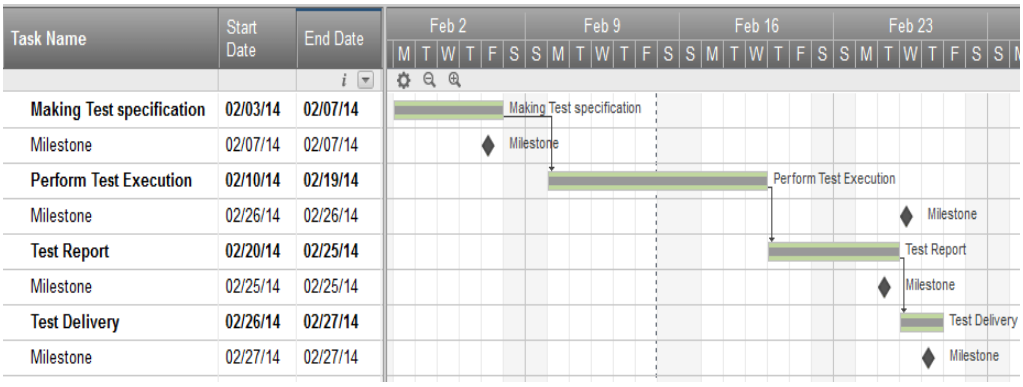
A testing environment is a setup of software and hardware on which the testing team is going to execute test cases.Test environment should describe how to set up test environment, resources like software, hardware, network and tools like test management tools, configuration & version management tools, automated testing tools, defect tracking tools.

⦿ Step 7) Schedule & Estimation

Schedule should contain a tasks with planned start & end dates. It should include tasks like, test plan preparation & approval, creation of test cases & reviews, execution of test cases, test summary report etc. you should include that estimation as well as the schedule to the Test Planning.

Task	**Members**	**Estimate effort**
Create the test specification	Test Designer	170 man-hour
Perform Test Execution	Tester, Test Administrator	80 man-hour
Test Report	Tester	10 man-hour
Test Delivery		20 man-hour
Total		**280 man-hour**

Example: Suppose the boss wants to complete the project in **one** month, you already estimated the effort for each tasks in Test Estimation. You can create the schedule as:



⦿ Step 8) Test Deliverables

Test Deliverables is a list of all the documents, tools and other components that has to be developed and maintained in support of the testing effort. There are different test deliverables at every phase of the software development lifecycle.

Test deliverables are provided **before testing** phase.

- Test plans document.
- Test cases documents
- Test Design specifications.

Test deliverables are provided **during the testing**

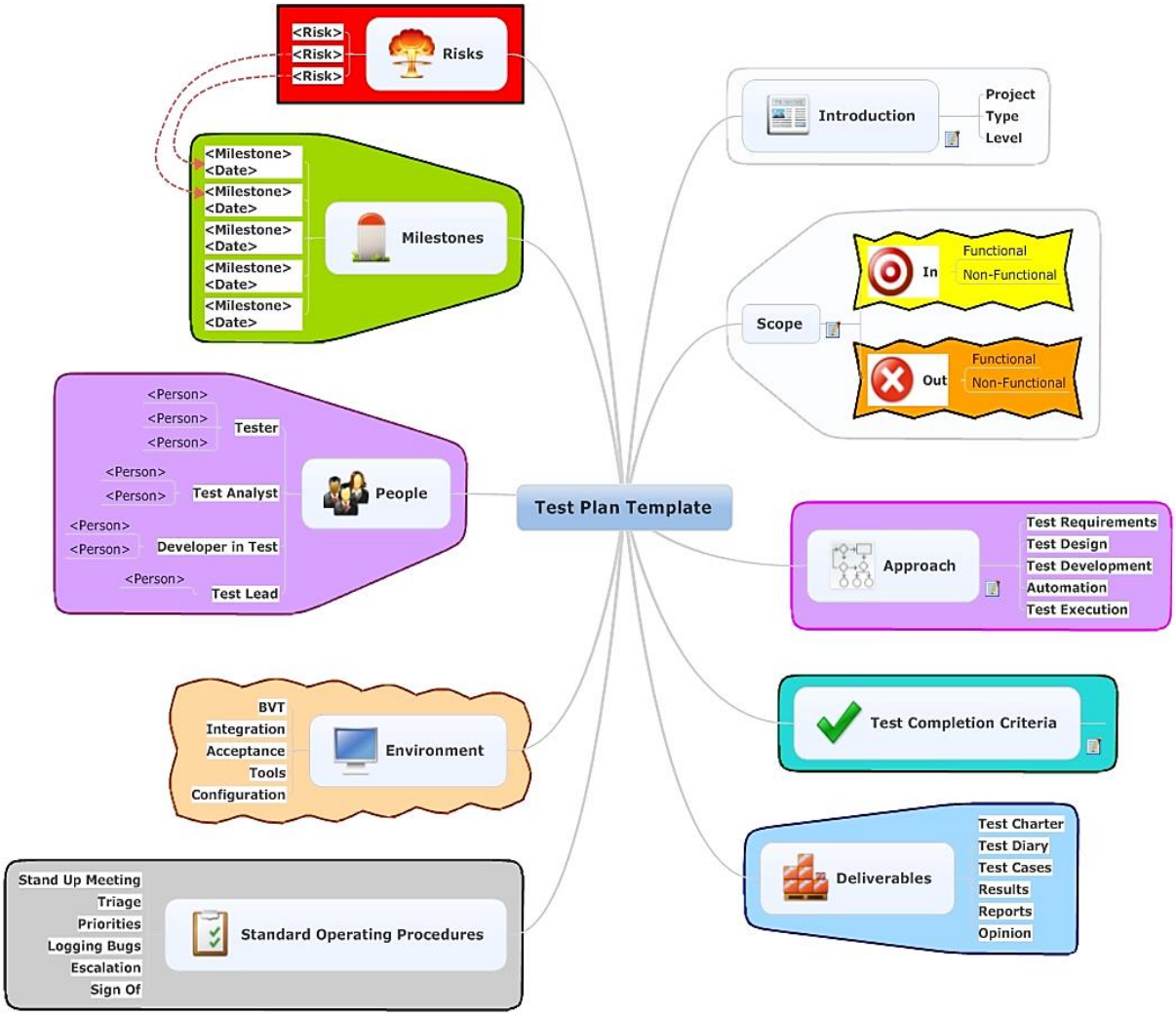
- Test Scripts
- Test Data
- Test Traceability Matrix
- Error logs and execution logs.

Test deliverables are provided **after the testing** cycles is over.

- Test Results/reports
- Defect Report
- Installation/ Test procedures guidelines

➢ Test Plan Template

The format and content of a software test plan depending on the processes, standards, and test management tools being implemented. IEEE standard for software test documentation, provides a summary of what a test plan can/should contain.



- **Test Plan Identifier:** Provide a unique identifier for the document.
- **Introduction:**
  - Provide an overview of the test plan.
  - Specify the goals/objectives.
  - Specify any constraints.
- **References:**
  - List the related documents:
    - Project Plan
    - Configuration Management Plan
- **Test Items:** List the test items (software/products) and their versions.
- **Features to be tested:** List the features of the software/product to be tested.
- **Features Not to Be Tested:**
  - List the features of the software/product which will not be tested.
  - Specify the reasons these features won't be tested.
- **Approach:**
  - Mention the overall approach to testing.
  - Specify the testing levels, the testing types, and the testing methods
- **Item Pass/Fail Criteria:**
  - Specify the criteria that will be used to determine whether each test item has passed or failed.
- **Suspension Criteria and Resumption Requirements:**
  - Specify criteria to be used to suspend the testing activity.
  - Specify testing activities which must be redone when testing is resumed.
- **Test Deliverables:**

- List test deliverables, and links to them if available, including the following:
  - Test Plan
  - Test Cases
  - Test Scripts
  - Defect/Enhancement Logs
  - Test Reports

- **Test Environment:**
  - Specify the properties of test environment: hardware, software, network etc.
  - List any testing or related tools.
- **Estimate:** Provide a summary of test estimates (cost or effort) and detailed estimation.
- **Schedule:** Provide a summary of the schedule, specifying key test milestones, and detailed schedule.
- **Staffing and Training Needs:**
  - Specify staffing needs by role and required skills.
  - Identify training that is necessary to provide those skills, if not already acquired.
- **Responsibilities:** List the responsibilities of each team/role/individual.
- **Risks:**
  - List the risks that have been identified.
  - Specify the mitigation plan and the contingency plan for each risk.
- **Assumptions and Dependencies:**
  - List the assumptions that have been made during the preparation of this plan.
  - List the dependencies.
- **Approvals:**
  - Specify the names and roles of all persons who must approve the plan.
  - Provide space for signatures and dates. (If the document is to be printed.)

Example: A Sample Test Plan Document for Internet Banking Application

1) Test Plan ID: IBS_ST_TP_001

2) Introduction: It is System Test Plan for Interment Banking System, internet web application, provides access to Account holders and guest users from any ware in the world. It has two interfaces one is Admin interface another is User interface. Admin can be accesses by Bank authorized users, user interface can be accessed by Bank account holders and guest users.

The propose of the system (Application) is to provide bank information and services online (through Internet), Bank account holders can get Banking services from any ware, without visiting the Bank branches.

3) Test Items:

Admin Interface:	**User Interface:**
Master Data User Management Reportsetc.	Information Personal Banking Corporate Banking Business etc.

4) References:

- Requirements
- Project Plan
- Test Strategy
- Use cases
- High Level and Low Level Design doc
- Process guide line doc
- Prototypes

5) Features to be tested:

a) Admin Interface:

i) Master Data	ii) User Management	iii) Reports
1) Add, Edit, Delete Branch 2) Add new ATM 3) Add new loan type 4) Add new account type 5) Add new deposit type etc.	1) Create new user 2) Edit user 3) Delete user etc.	1) Branch wise report 2) User wise report 3) Day, month, yearly reports 4) Service wise report (only loans, only new account. fixed deposits)

b) User Interface:

i) Information	ii) Personal Banking	iii) Corporate Banking
1) Branch locators 2) ATM locators 3) Loans information 4) Bank history 5) Bank financial details 6) Fixed deposits information	1)Login 2)Balance enquiry 3)Bill payment (utilities, subscriptions) 4)Fund transfer (transfer to same bank, other banks) 5) Statement generation (mini or detailed report).	1) Add, Edit, Delete user 2) Balance enquiry 3) Money transfer 4) Payroll 5) Reports etc.

6) Features not to be tested: NA

7) Entry Criteria:

a) Test Design	b) Test Execution:
Team formation, Responsibilities, Schedule, Requirements, Test Case Template etc. Training on Domain, on Automation tools	Readiness of Test Lab Readiness of AUT Requirements Test Case docs Test Data, Defect Report Template etc.

8) Exit Criteria:

- All possible test cases executed
- Maximum defects fixed, Final Regression performed successfully
- Confidence on Test process
- Time and Budget Limitations

9) Suspension Criteria:

- Show-Stopper bug found
- Supplier issues
- Vast changes in requirements
- If resolving defects are more

10) Roles & Responsibilities

Sno	Name	Role	Responsibilities	Remarks
1	Kumar	Test Lead	Test Planning, guidance, Monitoring and Test control	
2	Venkat Rao	Sr. Tester	Test Data Collection, Generating Test Scenarios.	
3	Swapna	Tester	Test Case Documentation, Test execution, defect reporting and tracking for Admin module.	
4	Srinivas	Tester	Test Case Documentation, Test execution, defect reporting and tracking for Personal Banking module.	
5	Suneetha	Tester	Test Case Documentation, Test execution, defect reporting and tracking for Corporate Banking module.	

11) Schedule:

Sno	Task	Days	Duration	Remarks
1)	Understanding & Analyzing Requirements	5	2nd July to 6th July	
2)	Review Meeting	01	9th July	
3)	Generating Test Scenarios.	10	11th July to 22nd July	
4)	Reviews	02	25th July to 26th July	
5)	Test Case Documentation	40	29th July to 12th August	
6)	Reviews	04	14th August to 18th August	
7)	Test Data collection	06	20th August to 26th August	

8)	Reviews	01	28th August	
9)	Verifying Test Environment setup	01	29th August	
10)	Create Test batches	02	30th, 31st August	
11)	Sanity Testing	01	3rd September	
12)	Comprehensive Testing	25	4th September to 2nd October	
13)	Sanity Testing	01	3rd October	
14)	Selecting Test cases	02	4th and 5th October	
15)	Regression Testing	05	8th October to12th October	
16)	Sanity Testing	01	15th October	
17)	Selecting Test cases	01	16th October	
18)	Regression Testing Cycle 2	04	17th October to 22nd October	
.	.	.	.	
28)	Final Regression	08	19th November to 28th November	
29)	Evaluating exit criteria	1 or 2	29th, 30th of November	
30)	Collecting all artifacts	02	3rd, 4th December	
31)	Test Summary Report	01	5th December	

Note: Regression Testing depends on Application and strength of Development team.

12) Training

- Training Program on Banking Domain
- Test Automation Training using QTP Tool

13) Risks & Mitigations

- Team members issues
- Vendor issues
- Time and Budget

14) Test Environment / Lab

Application Type: Web Application, Internet and Public

- **Server side:**
  - Windows 2003 Server or UNIX server
  - Ms Exchange server -a) Web server, b) EDP, c) Data storage
  - Bugzilla Tool
  - VSS
  - MS Office
  - QTP Tool
  - Browser IE 7.0
- **Client side:**
  - Windows XP + SP2
  - VSS
  - MS Office
  - QTP Etc.
- **AUT Environment**
  - .NET (C#, VC++, ADO)
  - IIS -web server
  - COM+ -App server
  - SQL Server 2005 for Database server

15) Test Deliverables

- Test Plan
- Review Reports
- RTM
- Test Scenario docs
- Test Case docs
- Test data
- Opened, Closed Defect Reports

- Test Summary Report

16) Approvals

SNO	Task/s	Author /Role	Date & Signature
1	Test Plan Documentation	Kumar (Test Lead)	
2	Review	Hari Prasad (QA Analyst)	
3	Approval	Vinod Rao (Project Manager)	

Example: Ecommerce Website Cross Browser Testing Test Plan

- **Test Plan Identifier:** Ecommerce Website Cross Browser Testing TP_1.0
- **References:** Project Plan, and Functional Specifications A1-C5.
- **Introduction :** This test plan for website cross browser testing supports the following objectives:

1. To define the tools to be used throughout the testing process.
2. To communicate to the responsible parties the items to be tested, set expectations, schedule and define environmental needs.
3. To define how tests will be conducted.

- **Test Items**

The system to be tested include the frontend customer facing website along with the backend ecommerce admin platform. These systems should be tested in Chrome 50.1, FireFox 45.0,Safari 9.1, and Internet Explorer 10 & 11.The system should be tested on both a Windows and Mac machine.

- **Features To Be Tested :** Features to be tested include the following
  - As a shopper, logging into the website as a shopper
  - As a shopper, navigating the store
  - As a shopper, adding items to a shopping cart
  - As a shopper, removing items from a shopping cart
  - As a shopper, purchasing multiple units of the same item
  - As a shopper, initiating a return
  - As a shopper, contacting support
  - As a shopper, completing an order cycle
  - As a shopper, cancelling an order
  - As a shopper, leaving a review
  - As an admin, granting a refund
  - As an admin, fulfilling an order
  - As an admin, answering a support inquiry
  - As an admin, moderating reviews
  - As an admin, validating in stock/outofstock
- **Features Not To Be Tested**

Mobile purchasing through a mobile device will not be tested. Only desktop web browser functionality will be tested.

- **Pass/Fail Criteria**

All core functionality of the systems should function perform correctly. There must be no critical defects found. An end user must be able to complete a purchase cycle successfully and initiate a refund without any errors. 95% of all test cases should pass and no failed cases.

- **Suspension Criteria**

Testing should be paused immediately if either system experiences login issues or failure in any basic CRUD (Create, Read, Update and Delete) actions.

- **Test Deliverables**

Upon completion, the test run results will be saved in Test Lodge and the test manager should then run a report for all completed tests.

- **Testing Tasks:** The following activities must be completed:
  - Test plan prepared.

- Functional specifications written and delivered to the testing team
- Environment should be ready for testing (test data, test logins, test payment information etc).
- Perform the tests.
- Prepare test summary report.

- **Environmental Needs**

The test site must be populated with test data including a variety of products with different prices. Test mode should be enabled for the backend ecommerce platform.

- **Responsibilities:** The Test Manager is responsible for facilitating the testing project, coordinating availability and schedule of testers and training them as needed. Each tester should understand theexpectations on completion date and level of quality. The Test Manager should alsocommunicate any risks to the team.
- **Staffing and Training Needs:** Testing should be done by two testers. Both testers should conduct testing on each system. The testers assigned should have basic knowledge of the ecommerce platform.
- **Schedule:** Testing will take place 4 weeks prior to the launch date. The first round of testing should be completed in 1 week.
- **Risks and Contingencies**

If the first round of testing is not completed within 1 week, it could delay bug fixes and final testing. If this happens, UAT would be pushed back and eventual affect the launch date. If the testers don't have a basic understanding the ecommerce platform, testing could be delayed or not conducted properly.

- **Approvals**

The test manager and product manage both must agree on completion of the testingproject anddetermine when it's ready to proceed to the next step.

Preparing Traceability Matrix (RTM)

Requirements Traceability Matrix is a document that maps or traces user requirement with test cases.i.e. Shows relationship between Requirements and Test Cases.It traces the requirements given by the client.This helps to ensure that all the requirements will be covered in the testing process and related test cases have been developed.Functionality should not missed.It is basically used to ensure that all requirements and Change Requests will be tested.

- **Requirement Traceability Matrix Parameters include**
  - ***Requirement ID:*** The requirement id is assigned to every requirement of the project.
  - ***Requirement description:*** for every requirement a detailed description is given in SRS (System/Software Requirement Specification) document.
  - ***Requirement Type:*** understand the type of requirements i.e. banking, telecom, healthcare, traveling, e-commerce, education, etc.
  - ***Test cases ID:*** Test cases are designed by the testing team. Test cases are also assigned with some ID.
  - Test Status , Priority
  - Bug Id etc.
- **Importance of RTM:**
  - Each test case is traced back to each requirement in the RTM. Therefore, there is less chance of missing any requirement in testing, and can 100% test coverage can be achieved.
  - RTM helps users to discover any change that was made to the requirements as well as the origin of the requirement.
  - It helps to keep a check between requirements and other development artifacts like technical and other requirements.
  - The Traceability matrix can help the tester identify whether by adding any requirement previous requirements are affected or not.

- RTM helps in evaluating the effect on the QA team to reuse the test case.

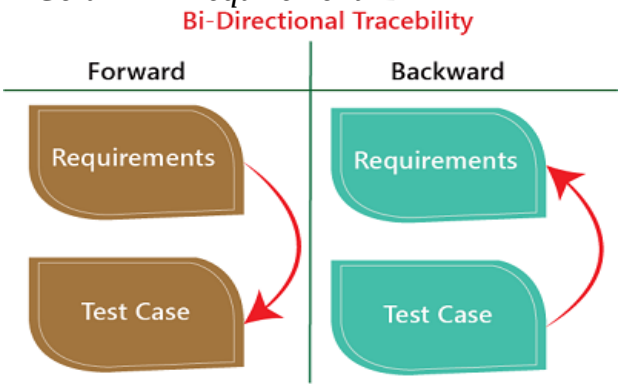
➢ **Types of Traceability Test Matrix**

- **Forward traceability**: verify that all requirements are covered in test cases and no functionality is missing in test cases. It helps you to ensure that all the requirements available in SRS can be traced back to test cases designed by the testers. It is used to check whether the project progresses in the right direction. It maps requirements with test cases. In forward traceability matrix:

  Rows = Requirement ID
  Column = Test case ID

- **Backward or reverse traceability:** It mapped the test cases with the requirements. We can verify that extra test case is not required as per our requirements. It helps you to ensure that any test cases that you have designed can be traced back to the requirements or user stories. You are not extending the scope of the work by just creating additional test cases that cannot be mapped to the requirement. The backward traceability matrix is also known as the **reverse traceability matrix**. In backward traceability matrix:

  Rows = Test cases ID
  Column = Requirement ID



- **Bi-directional traceability (Forward+Backward):** It is a combination of forwarding and backward traceability matrix, which is used to make sure that all the business needs are executed in the test cases. It also evaluates the modification in the requirement which is occurring due to the bugs in the application.
  Bi-directional traceability matrix = Forward traceability matrix + Backward traceability matrix

➢ **How to prepare Requirement Traceability Matrix (RTM):**

1. Collect all the available requirement documents.eg. Business Requirement Document(BRD), Functional Requirement Document(FSD), Technical Requirement Document(TSD)
2. Allot a unique Requirement ID for each and every Requirement
3. Create Test Cases for each and every requirement and link Test Case IDs to the respective Requirement ID.

➢ **How to create Requirement Traceability Matrix**

Requirement Traceability Matrix through a banking project.

Test cases of Business Requirement Document for banking project:

The customer should be able to login to banking website with the correct password and userid while manager should be able to login to the website through customer login page.

BR#	Module Name	Applicable Roles	Description
B1	Login and Logout	Manager Customer	**Customer:** A customer can login using the login page **Manager:** A manager can login using the login page f customer. Post Login homepage will show ifferent links based on role
B2	Enquiry	Customer	**ustomer:** A customer can have multiple bank accounts. He can view balance of his accounts only **Manager:** A manager can view balance of all the customers who come under his supervision
B3	Fund Transfer	Manager Customer	**Customer:** A customer can have transfer funds from his "own" account to any destination account. **Manager:** A manager can transfer funds from any

Business Requirement # for Guru99 banking project

Test cases of Technical Requirement Document (TRD) for banking project:

Login

T92 User-ID must not be blank
T93 Password must not be blank
T94 If userid and password are valid. Login

Here is our TRD (Technical Requirement Document)

QA teams don't document the BRD and TRD. **Function Requirement Documents (FRD)** is similar to Technical Requirement Document but the process of creating RTM remains the same.

Step 1: Test Case "Verify Login" when correct ID and Password is entered then login successfully.

TestCase #	Test Case	Test Steps	Test Data	Expected Result
1	Verify Login	1) Go to Login Page 2) Enter UserID 3) Enter Password 4) Click Login	id= Guru99 pass= 1234	Login Successful

When correct password and id entered, it should login successfully

Step 2: Identify the Technical Requirement when test case is verifying. For our test case, the technical requirement is T94 is being verified.

T94 If userid and password are valid. Login

T94 is our technical requirement that verifies successful login

Step 3: this Technical Requirement (T94) in the Test Case.

TestCase #	TR #		Test Steps	Test Data	Expected
1	T94	Verify Login	1) Go to Login Page 2) Enter UserID 3) Enter Password 4) Click Login	id= Guru99 pass= 1234	Login Successful

Note the Technical Requirement in the test case

Step 4: Identify the Business Requirement for which TR (Technical Requirement-T94) is defined

BR#	Module Name	Applicable Roles	Description
B1	Login and Logout	Manager Customer	**Customer:** A customer can login using the login page **Manager:** A manager can login using the login page of customer. Post Login homepage will show different links based on role

Identify the Business Requirement for which T94 is defined

Step 5: the BR (Business Requirement) in Test Case

TestCase #	BR #	TR #	Test Case	Test Steps	Test Data	Expe
1	B1	T94	Verify Login	1) Go to Login Page 2) Enter UserID 3) Enter Password 4) Click Login	id= Guru99 pass= 1234	Login Successful

Step 6: Do above for all Test Cases. Later Extract the First 3 Columns from your Test Suite. RTM in testing is Ready.

Business Requirement #	Technical Requirement #	Test Case ID
B1	T94	1
B2	T95	3
B3	T96	3
B4	T97	4

Requirement Traceability Matrix

- **Advantage of Requirement Traceability Matrix**
  - It confirms 100% test coverage
  - It highlights any requirements missing.
  - It shows the overall defects with a focus on business requirements
  - Gives Overview of all the requirements
  - Shows how requirements are linked to Test Cases
  - Easy to prepare
  - No special tool is required

Example: Project-Online Flight Booking Application

- **Business Requirement Document (BRD):** Suppose Flight Booking Application it shows below 2 requirements
  - **BR_1 Flight Booking Module:** allow user to book one or more tickets, one way or round way for future dates
  - **BR_2 Payment Module:** to make payment for booked tickets via Credit / Debit Card or through Reward Points
- **Functional Specification Document (FSD):** This document is prepared by Technical team which further detailed business requirements into functional requirements that can be implemented in software.

Suppose above 2 business requirements in BRD have more detailed functional requirements:

BR_1 Reservation Module:

- **FR_1 : One Way Ticket booking :** allow user to book one way ticket
- **FR_2 Round Way Ticket :** allow user to book round way ticket
- **FR_3 Multicity Ticket booking** allow user to book one way or round way ticket for multiple cities

BR_2 Payment Module:

- **FR_4: By Credit Card:** allow user to make payment by Credit Cards
- **FR_5 By Debit Card:** allow user to make payment by Debit Cards
- **FR_6 By Reward Points:** allow user to make payment by Reward Points

<u>Written some test cases or test scenarios for each functional requirement.</u>

Test Case ID#	Test Case Description
TC#001	Verify if user is able to book one way ticket
TC#002	Verify if user is able to book multiple one way tickets
TC#003	Verify if user is able to book round way ticket
TC#004	Verify if user is able to book multiple round way tickets
TC#005	Verify if user is able to book one way ticket for multiple cities
TC#006	Verify if user is able to book round way ticket for multiple cities
TC#007	Verify if usre is able to pay by Master Card
TC#008	Verify if usre is able to pay by Visa Card
TC#009	Verify if usre is able to pay by Debit Cards
TC#010	Verify if usre is able to pay Fully by Reward Points
TC#011	Verify if usre is able to pay partially by Reward Points

Prepare simple Requirements Traceability Matrix (RTM) for above example it would like as below:

REQUIREMENTS TRACEABILITY MATRIX					
Project Name: Online Flight Booking Application					
Business Requirements Document BRD		Functional Requirements Document FSD			Test Case Document
Business Requirement ID#	Business Requirement / Business Use case	Functional Requirement ID#	Functional Requirement / Use Case	Priority	Test Case ID#
BR_1	Reservation Module	FR_1	One Way Ticket booking	High	TC#001 TC#002
		FR_2	Round Way Ticket		TC#003 TC#004
		FR_3	Multicity Ticket booking	High	TC#005 TC#006
BR_2	Payment Module	FR_4	By Credit Card	High	TC#007 TC#008
		FR_5	By Debit Card	High	TC#009
		FR_6	By Reward Points	Medium	TC#010 TC#011

You can also add Execution Status and Defects columns in RTM to view overall status of all requirements along with Test Cases.

Example: Consider an example of a retail application for online shopping. The goal is to make the user's purchase experience as simple as possible. It requires a login module, product listing module, and payment gateway module. Requirement traceability sample for reference:

BR ID#	BR Use Case	FR ID#	FR Use Case	Priority	TC ID#	Status	Comments
BR_001	Login	FR_001	With Mobile#	High	TC_001 TC_003 TC_004	Finished	Jan 1: Testing Started Jan 6: Defect Reported Jan 10: Defect Fixed Jan 12: Pass
		FR_002	With Email Id	High	TC_002 TC_003 TC_004	In Progress	Jan 10: Testing Started

Creating a Requirement Traceability Matrix

The following steps are required to create a requirements traceability matrix:

1. Define Objectives and Milestones
2. Gather Required Documents
3. Create a Traceability Matrix
4. Validate the Final Result

1. Define Objectives and Milestones

You can define these modules in the RTM as below:

- **BR_001 Login Module:** It should allow users to log in.
- **BR_002 Product Listing Module:** It should allow users to browse different products with advanced filters.
- **BR_003 Payment Gateway Module:** It should allow users to make payments through wallets, credit cards, debit cards, etc.

2. Gather Required Documents

Gather all required documents for building an effective traceability matrix. Documents include Business Requirements Document (BRD), Functional Requirements Document (FRD), Technical Requirements Document (TRD), and Test Case Scenarios Documents as prepared by the testing team. Functional requirements, which are as follows:

<u>BR_001 Login Module:</u>

- **FR_001 With Mobile Number:** It should allow users to sign up with a mobile number and OTP.
- **FR_002 With Email ID:** It should allow users to sign up with an email id.

<u>BR_002 Product Listing Module:</u>

- **FR_003 Sort by:** It should allow users to sort products by different prices, customer ratings, popularity, discounts, etc.
- **FR_004 Filters:** Based on the size, color, and brand, users should be able to apply filters.

BR_003 Payment Gateway Module:

- **FR_005 By Wallet:** It should allow users to make payments with wallets like google pay, Paytm.
- **FR_006 By Credit Card:** It should allow users to make payments with different credit cards.
- **FR_007 By Debit Card:** It should allow users to make payments with different debit cards.

3. Create a Traceability Matrix

Test Case ID	Test Case Description
TC_001	Verify if user can log in with mobile number
TC_002	Verify if user can log in with email id
TC_003	Verify if user can order single product
TC_004	Verify if user can order multiple products
TC_005	Verify if user can apply single or multiple filters
TC_006	Verify if user can sort products with different criteria
TC_007	Verify if user can make payment using wallets
TC_008	Verify if user can make payment using credit card
TC_009	Verify if user can make payment using debit card

Business Requirement Document (BRD)		Functional Requirement Document (FRD)		Test Case Document			
BR ID#	BR Use Case	FR ID#	FR Use Case	Priority	TC ID#	Status	Comments
BR_001	Login	FR_001	With Mobile#	High	TC_001 TC_003 TC_004	Finished	Jan 1: Testing Started Jan 6: Defect Reported Jan 10: Defect Fixed Jan 12: Pass
		FR_002	With Email Id	High	TC_002 TC_003 TC_004	In Progress	Jan 10: Testing Started
BR_002	Product Listing	FR_003	Sort By	High	TC_006	Finished	Jan 1: Testing Started Jan 6: Defect Reported Jan 10: Defect Fixed Jan 12: Pass
		FR_004	Filters	High	TC_005	Finished	Jan 1: Testing Started Jan 6: Defect Reported Jan 10: Defect Fixed Jan 12: Pass
BR_003	Payment Gateway	FR_005	With Wallet	Medium	TC_007	Not Started	
		FR_006	With Credit Card	High	TC_008	In Progress	Jan 12: Testing Started
		FR_007	With Debit Card	High	TC_009	In Progress	Jan 12: Testing Started

As shown above, a requirement traceability matrix will:

- Show the required coverage in test cases
- Design and Execution status of each test case
- If there is any User Acceptance Testing (UAT), then UAT status can also be captured on the same matrix.

So, a requirement traceability matrix is a one-stop shop for all requirements till user acceptance.

#4. Validate the Final Result

Once the requirements traceability matrix is created, you must verify that all modules and test scenarios are developed and tested.

Example

Requirements Traceability Matrix

Project Name: Beta Insurance
Project Number: 179643
Project Manager: Rishi Joel

Purpose: To mange requirements throughout the software development lifecycle. The Traceability Matrix ensures that requirements are captured in the design, implemented in the code, and verified by testing.

Use Case	Requirement Number	Requirement Description	Status	Release	Test Case	Design Component / Module	Comments
Login of the application	BI_001	Login of the web application should be enabled	Approved	MVP	Check login of web both postive and negative	Login Module	
Login of the application	BI_002	Login of the Android app should be enabled	Proposed	V1	Check login of Android app both postive and negative	Login Module	
Login of the application	BI_003	Login of the iOS app should be enabled	Proposed	V2	Check login of iOS app both postive and negative	Login Module	

📢 Writing Test Execution Report

Test execution report contains the execution detail and the result of test execution. This report shows the state of the test execution and test progress to higher management by testing team. When we deliver the software to client, we shall send the full test execution detail and coverage in brief. Test execution is the process of executing the code and comparing expected and actual results. Test execution report can be sent on a daily basis or weekly basis. The test execution report shows number of test cases passed, failed, blocked or not executed. It also list the bugs faced during the test execution phase.

Element	Description
Test item	Item, module or software that is been testing.
Scenario	Scenario of the test.
Responsible tester	Who tested the particular test item
Date of test execution	Date of the test executed.
Status	Status of test execution.
Test environment	Detail of test environment. Detail of hardware, software & tools.
Risk and constraint	Any risk that impact the test execution and any constrain.
Test execution summary	Provide detail of test execution , test type, defect rage etc.
Defect summary details	Defect count. Rate of open and closed defect. Severity and status wise defects.
Test cases detail	Planned and executed test cases. Status, result of execution and test case.
Test progress	Over all progress of test and which cycle the test execution is going on.

Different reports generally used in testing by testers.

➢ Daily Status Report

Daily status reports that contains information about the different tasks performed or executed, issues logged and difficulty faced by a tester. Every testing team member sends daily status report to their test manager at the end of each day.

Three primary testing reports are frequently used by testers:

- **Test Plan status report –** shows the current status of the test plan. This report notifies to team, whether any changes are being made to the existing test plan or any new version is introduced.
- **Test Documentation Status –** shows the status of the test cases and test scenarios.
- **Test Execution Status –** shows what the status of testing and how stable the overall application.

The basic components of test execution report can be broadly classified into 3 categories,

⇒ **Test Report Summary -** gives a brief overview of the overall execution status.

- The number of test cases planned within the cycle.
- The number of test cases executed within the cycle.
- The total number of test cases executed till date.
- Defects encountered within the cycle.
- Total defects encountered till date.
- Pass percentage and defect density.

This report helps the project manager and product owner get a quick overview of the current execution and analyze it to predict any blockage that might arise during the testing cycle

Project Name : Banking Application	
Overall progress of the QA cycle	**On time**
Total number of test cases	100
Number of testers	5
Test cycle duration	5 days
Status for 10/27/2017	
Number of test cases planned	20
Number of test cases executed	18
Number of test cases executed overall	78
Number of defects encountered today	2
Number of defect encountered so far	10
Number of critical defects- still open	3
Overall status	
Number of test cases planned	100
Number of test cases executed	78
Pass Percentage of the defects	98%
Defects density	2.5 per day
Critical defects percentage	20%

Figure: Test Summary Report

⇒Test Scenario/ Case status

The test scenario/ test case status gives a detailed report of the status of the test scenarios. It enlists the different scenarios and test cases executed during the testing cycle, along with their corresponding status. You can even find the name of the testers who executed the test cases along with a brief description of the issues which were encountered during test execution.

Module	Scenarios	Sub Levels	Complexity	Responsible tester	Date of Execution(Can be past, present or future date)	Status(Pass/ Fail/blocked/not executed)	Defect ID- Brief description	Severity	Status
Blog	Blog entries	Content	Medium	Tester A	10/27/2017	Pass			
Blog	Social Widgets	Facebook	Complex	Tester B	10/27/2017	Pass			
		Twitter	Complex	Tester B	10/27/2017	Pass			
		Linkedin	Medium	Tester B	10/27/2017	Pass			
E-course	Course management	Add course	Complex	Tester A	10/27/2017	Pass			
		Make payment	Complex	Tester A	10/27/2017	Pass			
		Verify enrollment	Medium	Tester A	10/27/2017	Pass			
E-course	Resume ecourse	Verify e-course session management	Complex	Tester B	10/27/2017	Fail	#4573 - Session data doesn't persist on different tabs	1- High	Open
		Managing courses	Complex	Tester B	10/27/2017	Blocked	1028		
		Verify course content	Medium	Tester B	10/27/2017	Not executed			
E-course	Sharing course	Adding course	Complex	Tester X	10/27/2017	Pass			
		Sharing course	Complex	Tester X	10/27/2017	Pass			
		Selecting social widget	Medium	Tester X	10/27/2017	Fail	#5655 - Wdgets not	2- Medium	Assigned

⇒ Reported bugs

Any bugs which were reported during the test cycle are mentioned in this section.

Defect ID	Defect Description	Steps to reproduce	Severity	State	Created by	Assigned to
5655	Widget not visible on UI	1. Login to the site using the admin credentials 2. Click on E-course management. 3. Select e-course 4. Share e-course using social media widgets Expected result: The widgets should be visible on each e-course page Actual Result: Widget not visible	1- High	Open	Tester X	Dev representative

Figure: Reported bugs

Test Summary Report

Test summary report is a document which contains summary of test activities and final test results. Client and Project management know the status of testing and Application Quality. This summary report is like to Test Closure phase of STLC.

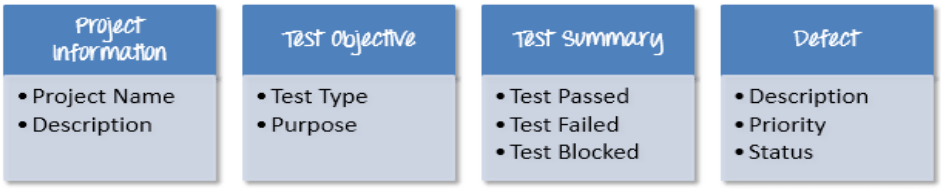
Features of test summary report:

- Assessment of how well the Testing is performed during the testing process.
- Summarizes the results of all testing efforts.
- Client can evaluate the quality of the tested product and take a decision on the software release.
- Test managers prepare this document at the end of the testing project.

Test summary report will be different type for different types of testing.

- Risk based testing report should contain the test coverage and unresolved defects against the product risks.
- Requirements based testing should measure coverage against the requirements.

➢ **Contents of a Test summary report:**



- Name of project
- Release No.
- Reference documents: Reference can be given of test planning document or test strategy document or project plan
- Target Audience: Who all will be the recipients of this report
- Test Cycle:

➢ **Testing Type / Level:** *Mention testing type / level like unit testing or system testing or integration testing or it may be a functional testing or performance testing etc*

➢ **Test Suite Data:**

- Number of test suites planned.
- No of test suites executed
- No of test suites could not be executed

➢ **Test Case Data:**

- Number of test cases planned.
- Number and percentage of test cases implemented.
- Number and percentage of test cases executed.
- Number and percentage of test cases passed.
- Number and percentage of test cases failed (total and by severity).

➢ **Defect Data:**

- Total open bugs
- New bugs found
- Open bugs of previous release
- Reopened bugs
- Bugs marked as "Not a Bug"
- Bugs marked as "Deferred"

➢ **Risks / open issues / support required:** Mention here all potential risks and current open issues with assignment of action items to specific stakeholder. Mention the details of support required, if any.

➢ Some other information can be part of test summary report:

i) Testing Members
ii) Objectives
iii) Scope
iv)Client Details
v) Test Approach used
vi)Test Results

vii) Defect Report

Example: A Sample Test Summary Report

Introduction: It is for Internet Banking System Application Version 2.0, IBS (Internet Banking System) has Information, Personal banking, Corporate Banking, Master Data, User Management and Reports modules already. Now in this release some features added to Personal Banking Module and One New module "Small Business" added.

Test Items:

- Personal Banking
- New Business
- Reports

Reference documents:

- Test Plan
- Test Case documents
- Opened and Closed Defect Reports
- Metrics docs
- Review Reports.

Target Audience:

- Project Manager
- Release Team
- Maintenance Team and Customer (End Users).

Test Summary

a) Test Suite Information:

- Test suites planned: 42
- Test suites implemented: 39
- Test suites executed: 38

b) Test Case Information:

- Test cases planned: 983
- Test cases implemented: 978
- Test cases executed: 967
- Test cases passed: 940 (Final)
- Test cases failed: 14 (Final)
- Test Cases pending: 3

c) Defect Report Information

- Defects found in comprehensive Testing: 148
- Test Cases selected for Regression Testing Cycle1: 320
- Defects found in Regression Testing Cycle1: 96
- Test Cases selected for Regression Testing Cycle 2: 213
- Defects found in Regression Testing Cycle 2: 42
- Test Cases selected for Regression Testing Cycle 3: 107
- Defects found in Regression Testing Cycle 3: 26
- Test Cases selected for Final Regression: 512
- Number of opened defects in this release: 14

Approvals

Name	Role	Responsibility	Date & Signature
Abhi Rao	Test Lead	Test Summary Report preparation	20-06-2012
Suyogkumar	QA Analyst	Review	25-06-2012
Murali	Project Manager	Approval	29-06-2012

Test Scenarios for Simple Calculator

Write the test cases based on the following functions and Test scenarios.

1. Check the calculator if it starts by on the button.
2. Check if all the numbers are working (0 to 9)
3. Check if the arithmetic keys (+, -, *, %, /) are working.
4. Check if the clear key is working.
5. Check if the sum key is working.
6. Check if the Subtraction key is working.
7. Check if the Multiplication key is working.
8. Check if the Division key is working.
9. Check if the square root keys are working.
10. Check if the Percentages key is working.

Functionality Test Cases For Calculator

1. Check the addition of two integer numbers.
2. Check the addition of two negative numbers.
3. Check the addition of one positive and one negative number.
4. Check the subtraction of two integer numbers.
5. Check the subtraction of two negative numbers.
6. Check the subtraction of one negative and one positive number.
7. Check the multiplication of two integer numbers.
8. Check the multiplication of two negative numbers.
9. Check the multiplication of one negative and one positive number.
10. Check the division of two integer numbers.
11. Check the division of two negative numbers.
12. Check the division of one positive number and one integer number.
13. Check the division of a number by zero.
14. Check the division of a number by a negative number.
15. Check the division of zero by any number.
16. Check if the functionality using BODMAS/BIDMAS works as expected.

Test Case -ID	Test case	Pre-condition	Steps	Test data	Expected Result	Actual Result	Status
TC1	To add two integer and display the result on ten digit calculation	Calculator is switched on	1. Press Key in a valid integer from - 9999999999 to +9999999999 2. Key in operator + 3. Press Key in second operand, a valid integer from - 9999999999 To +999999999	135 + 100	235(addition, above ten digits will be expressed in exponential form)	235	Pass
TC-2	To subtract two integer and display the result on ten digit calculator	Calculator is switched on	1. Press Key in a valid integer from - 9999999999 to +9999999999 2. Key in operator - 3. Press Key in second operand, a valid integer from - 9999999999 To +999999999	135-100	35(subtraction, above ten digits will be expressed in exponential form)	35	Pass

TC-3	To multiply two integer and display the result on ten digit calculator	Calculator is switched on	1. Press Key in a valid integer from - 9999999999 to +9999999999 2. Key in operator x 3. Press Key in second operand, a valid integer from - 9999999999 To +999999999	100 x 400	40000(multi plication, above ten digits will be expressed in exponential form)	40000	Pass
TC4	To divide two integer and display the result on ten digit calculator	Calculator is switched on	1. Press Key in a valid integer from - 9999999999 to +9999999999 2. Key in operator / 3. Press Key in second operand, a valid integer from - 9999999999 To +999999999	100 / 25	40(division, above ten digits will be expressed in exponential form)	40	Pass
TC5	To clear the screen	Calculator is switched on	Press C		Symbol _0‘ should appear on screen	Symbol _0‘ appear s on screen	Pass
TC6	To delete digits one by one	Calculator is switched on	Press <- (backspace)		One Digit should be deleted from right hand side	One Digit is deleted from right handside	Pass

Test Case for Online Flight Reservation System

1. Check whether the page is possible to login without providing the user with any details
2. Check and login to the page with a valid username and password
3. Check and log in to the page with the correct username and invalid password
4. Check whether the password is encrypted form while entering the same
5. Check whether the application is accepting the date in DD/MM/YY format
6. Check the date by entering the same in MM/DD/YY but execute the test by providing the single-digit value for the date and month E.g.6/7/13
7. Check the website whether there is any functionality for adding new flights
8. Check after adding a new plane in the system if the trip has all the details like flight name, code of the flight, from and to destinations, frequency, timings, and capacity of the plane.
9. Check you can search the newly added flight by entering the name, from to airport details, or by entering the flight code
10. Check whether the drop-down list box are listings the name of the boarding places
11. Check whether the drop-down list box is listing the name of the destination places to select the location.
12. Check the application whether it is showing the availability of the flights with the date and timings of the departure
13. Check is the search result has the details like and availability of seats
14. Check if the user clicks on the flight name is it giving all the details
15. Check is the user is getting the real-time results of the flight like the availability of seats

16. Check when the user clicks on the book button then is the graphical user interface for the seats is displaying
17. Check if the pricing of different types of seats is shown to the user
18. Check if the user can select one or more seats.
19. Check whether the user can choose to pre-book a seat or not, the vendor used to try to choose a booked seat that time it should display a warning message
20. Check after selecting one or multiple seats, when the user clicks the process button that time it redirects the user to the passenger details page where the user needs to feel all the passenger details to complete the booking process.
21. Check a user can download that ticket the ticket is visible after completing the booking process
22. Check the seat status after a user is or completes the booking process, the status should be changed from available to book
23. Check after completing the booking process if the user is getting an email confirmation to the email ID provided on the passenger details page
24. Check how many seats a user can book at a time, If he's tried to book more than the allowed seats then an error message should be display
25. Check whether all the different payment options are working properly
26. Check while booking seats. Additional information should also be displayed like luggage commas extra legroom, food, etc. if the user selects any extra option, then the extra charges also need to be added to the actual price of the seat.
27. Check a user can cancel a book ticket after canceling the ticket the money should be refunded to the same account which is used while booking a ticket
28. Check if a seat is canceled then the status of the seat should change from booked to available
29. Check is the help option is provided on the website or portal, also verify is the help section contains all the required information
30. Check in the contact up page heaven option to contact Airlines for any assistance.
31. Check if is there any option for web check-in before the flight
32. Check if is there any functionality to inform a user if the flight gets delayed
33. Check whether the cursor is moving to the name of the passenger field for entering the passenger name who is going to travel
34. Check the application is allowed to select the class of travel by clicking the available options like first, second and business class
35. Check whether the application is showing the details of the passenger order once it has been placed.

Test Cases for social networking sites like Whatsapp

1. Verify that on downloading the Whatsapp application, users can register using a new mobile number.
2. Verify that for a new mobile number user will get a verification code on his mobile and filling in the same verifies the new user account.
3. Check the maximum number of incorrect attempts allowed while filling out the verification code.
4. Verify that registering an existing mobile number for new user account registration is not allowed.
5. Verify that on successful registration all the contacts in the user's contact directory get imported to the Whatsapp contact list.
6. Verify that the user can set DP and status on Whatsapp.
7. Verify that the user can update the existing DP and Whatsapp status.
8. Verify that the user can send messages to any individual selected from his contact list.
9. Verify that 'Chats' window contains all the chat list with DP and name and last message preview of the other person with whom chat was initiated.

10. Verify that clicking a chat in the chat list opens a new window containing all the chats received and sent with the other person.
11. Verify that the user can check the message delivered and read the time for a message in the 'Message Info' section.
12. Verify that the user can share or receive contact with the other person.
13. Verify that the user can create a group by adding multiple people from his contact list.
14. Verify that the user can send and receive the message in group chats.
15. Verify that users can send and receive images, audio, video, and emoticons in the chat with individuals.
16. Verify that users can send and receive images, audio, video, and emoticons in group chats.
17. Verify that the user can send and receive chats in the secondary languages available.
18. Verify that users can delete text, images, audio, and video messages within a chat.
19. Verify that users can clear their complete chat history in an individual or group chat.
20. Verify that users can archive chats in an individual or group chat.
21. Verify that users can block a user to prevent any message from getting received from the blocked contact.
22. Verify that the user makes WhatsApp calls to the person in his contact list.
23. Verify that the user can receive WhatsApp calls from the person in his contact list.
24. Verify that users can mark chats as favorites and access all chats marked as favorites from the 'Favorites' section.